Cheatsheet · 8 chapters on one page

Deep Learning cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

48 key ideas | 149 formulas | 31 tables | 32 tips and traps

STEP 1 Neural network basics

1.1 Neural networks: from perceptrons to MLPs

Neurons, the perceptron and XOR, multilayer networks, activations and scikit-learn MLPs.

KEY IDEAS

- **Neuron** $a = \varphi(w \cdot x + b)$. Layer: $h = \varphi(Wx + b)$. Parameters per layer $n_{out}(n_{in} + 1)$.
- **Perceptron** Step output, update $w \mathrel{+}= \eta(y - \hat{y})x$ on mistakes. Converges if separable, $(R/\gamma)^2$ mistakes. Fails on XOR.
- **MLP** Hidden layers with nonlinear activations learn features. Without φ layers collapse to one linear map.
- **Universal approximation** One hidden layer, enough units, any continuous function on a bounded set. Depth is more efficient.
- **Activations** ReLU or GELU hidden. Sigmoid slope ≤ 0.25, saturates. Softmax for classes. Dead ReLU, leaky ReLU.
- **In practice** Scale inputs, MLPClassifier(hidden_layer_sizes, alpha), compare with logistic regression, watch train and test gap.

FORMULAS

Neuron — Exam favourite

$$a = \varphi(\mathbf{w} \cdot \mathbf{x} + b)$$

φ is the activation function.

Lesson: From one neuron to the perceptron

Perceptron rule — Exam favourite

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}, \quad b \leftarrow b + \eta(y - \hat{y})$$

Only wrong predictions change anything.

Lesson: From one neuron to the perceptron

Convergence bound — Exam favourite

$$\text{mistakes} \le (R / \gamma)^2$$

R: largest input size, γ: margin of the best separating line (Novikoff 1962).

Lesson: From one neuron to the perceptron

Layer — Exam favourite

$$\mathbf{h} = \varphi(W\mathbf{x} + \mathbf{b})$$

W has one row per neuron.

Lesson: Multilayer networks and why hidden layers matter

Two layer network — Exam favourite

$$\hat{y} = \sigma(\mathbf{w}_2 \cdot \tanh(W_1\mathbf{x} + \mathbf{b}_1) + b_2)$$

The network used for XOR.

Lesson: Multilayer networks and why hidden layers matter

Why nonlinearity — Exam favourite

$$W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$$

Without φ, two layers equal one.

Lesson: Multilayer networks and why hidden layers matter

Sigmoid slope

Exam favourite

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le 0.25$$

Largest at z = 0.

Lesson: Activation functions

Tanh slope

Exam favourite

$$\tanh'(z) = 1 - \tanh^2(z)$$

Equals 1 at z = 0.

Lesson: Activation functions

Softmax

Exam favourite

$$p_i = e^{z_i} / \Sigma_j e^{z_j}$$

Subtract max z first for numerical safety.

Lesson: Activation functions

Neuron

Exam favourite

$$a = \varphi(\Sigma_i w_i x_i + b)$$

Weighted sum, then activation.

Lesson: The maths, all in one place

Layer in matrix form

Exam favourite

$$\mathbf{h} = \varphi(W\mathbf{x} + \mathbf{b})$$

Shape: W is (units out) by (units in).

Lesson: The maths, all in one place

Number of parameters

Exam favourite

$$n_{out}(n_{in} + 1) \text{ per dense layer}$$

Weights plus one bias per unit.

Lesson: The maths, all in one place

Binary cross entropy

Exam favourite

$$L = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$$

Loss for a sigmoid output.

Lesson: The maths, all in one place

Universal approximation

$$|f(\mathbf{x}) - \Sigma_j v_j \sigma(\mathbf{w}_j \cdot \mathbf{x} + b_j)| < \varepsilon \text{ on a bounded set}$$

Cybenko (1989): one hidden layer suffices, given enough units.

Lesson: The maths, all in one place

ReLU

$$\max(0, z), \text{ slope 1 for } z > 0$$

No vanishing for positive inputs.

Lesson: The maths, all in one place

TABLES

Table: activation · formula · range · use

Activation	Formula	Range	Use
Sigmoid	$1 / (1 + e^{-z})$	(0, 1)	Binary output, gates in LSTMs
Tanh	$(e^{z} - e^{-z}) / (e^{z} + e^{-z})$	(−1, 1)	Recurrent networks, small MLPs
ReLU	max(0, z)	[0, ∞)	Default for hidden layers
Leaky ReLU	max(0.01z, z)	(−∞, ∞)	When many ReLUs die
GELU	$z\,\Phi(z)$	about (−0.17, ∞)	Transformers such as BERT and GPT
Softmax	$e^{z_i} / \Sigma_j e^{z_j}$	Probabilities summing to 1	Multi class output

Lesson: Activation functions

Table: setting · default · what it does

Setting	Default	What it does
hidden_layer_sizes	(100,)	Units in each hidden layer
activation	relu	relu, tanh, logistic or identity
solver	adam	adam, sgd or lbfgs (good for small data)
alpha	0.0001	L2 penalty on the weights
learning_rate_init	0.001	Step size for adam and sgd
early_stopping	False	Hold out 10% and stop when it stops improving

Lesson: MLPs in practice with scikit-learn

Table: · perceptron · logistic regression · mlp

	Perceptron	Logistic regression	MLP
Output	Hard 0 or 1	Probability	Probability
Boundary	Straight line	Straight line	Any shape, given enough units
Learning	Fix mistakes only	Gradient descent on cross entropy	Backpropagation on cross entropy
Converges	Only if separable	Yes, convex loss	To a local minimum, loss is not convex
XOR	Cannot	Cannot	Can

Lesson: Perceptron, logistic regression and MLP compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1486	XOR Operation in an Array	Easy	Build each number and fold them with XOR	The numbers are start, start + 2, start + 4 and so on. XOR them one by one. XOR is the function a single perceptron cannot learn, so it is worth knowing well.
2433	Find The Original Array of Prefix Xor	Medium	XOR neighbouring prefixes to undo them	Because a ^ a = 0, the original value at i is pref[i] ^ pref[i − 1]. Undoing a running combination step by step is the same idea as peeling layers back during backpropagation.
1442	Count Triplets That Can Form Two Arrays of Equal XOR	Medium	Equal XOR halves mean the whole range XORs to 0	a == b exactly when arr[i] ^ … ^ arr[k] is 0, which means two prefix XORs are equal. For each such pair of prefix positions, every split point in between works. Turning a condition into a simpler hidden feature is what hidden layers do.
421	Maximum XOR of Two Numbers in an Array	Medium	Decide the answer one bit at a time, from the top	Try to set each bit of the answer from the highest down. Keep the set of prefixes of the numbers and check whether two prefixes XOR to the answer you want. Building a result greedily, bit by bit, mirrors how a deep network builds features layer by layer.
1707	Maximum XOR With an Element From Array	Hard	Sort queries by their limit and grow a binary trie	Answer queries in order of their limit m, inserting every number up to m into a trie of bits. For each query walk the trie, choosing the opposite bit whenever possible. Processing inputs in a clever order so that one structure serves every query is a common trick in efficient training pipelines.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

On 40 points that a line can separate, the perceptron makes no mistakes after epoch 6. It learns AND in 6 epochs and OR in 4, but after 100 epochs on XOR it still predicts [1, 1, 0, 0] for targets [0, 1, 1, 0]. Minsky and Papert's 1969 book made this limit famous.

Lesson: From one neuron to the perceptron

IN THE EXAM A network with 3 tanh hidden units learns XOR: the loss falls from 0.7471 to 0.0018 in 3000 steps and the outputs become 0, 0.998, 0.998 and 0.003. The hidden values show what happened: each input point got new coordinates in which a single line separates the two classes.

Lesson: Multilayer networks and why hidden layers matter

IN THE EXAM Sigmoid's slope is 0.25 at 0 and 0.007 at ±5. Tanh's slope is 1 at 0 but 0 at ±5 (to three decimals). ReLU's slope is exactly 1 for every positive input. Multiplying ten sigmoid slopes of at most 0.25 can shrink a gradient by a factor of about one millionth.

Lesson: Activation functions

IN THE EXAM On two noisy half moons, logistic regression reaches 0.861 test accuracy. An MLP with 2 hidden units (9 weights) does no better, 0.856. With 32 units it reaches 0.883, and with two layers of 32 (1185 weights) 0.900, while training accuracy climbs to 0.952. More capacity fits the curve better, but the gap between train and test grows too.

Lesson: MLPs in practice with scikit-learn

1.2 Backpropagation and automatic differentiation

The chain rule on graphs, autodiff, matrix backprop with gradient checks, and gradient flow through depth.

KEY IDEAS

- **Backpropagation** Chain rule backwards on the graph. Store forward values. Add passes, multiply swaps, $\sigma' = \sigma(1 - \sigma)$.
- **Accumulation** A value used twice gets the sum of both gradients. Zero gradients before each step.
- **Autodiff** Reverse mode: all gradients for about 2 to 3 forward costs. Forward mode: one pass per input.
- **Matrix gradients** $\partial L/\partial W = X^T G$, $\partial L/\partial b$ = row sum of G, $\partial L/\partial X = GW^T$. Softmax CE: $p - y$.
- **Gradient check** Central difference with $h \approx 1e{-}5$, relative error below $1e{-}7$.
- **Gradient flow** Vanishing and exploding from products of slopes. He $2/n_{in}$ for ReLU, Xavier for tanh, clip by norm.

FORMULAS

Chain rule Exam favourite

$$\partial L/\partial x = (\partial L/\partial u)(\partial u/\partial x)$$

Multiply local slopes along the path.

Lesson: The chain rule on a computational graph

Add and multiply gates Exam favourite

$$u = a + b: \partial u/\partial a = 1. \quad u = ab: \partial u/\partial a = b$$

Add passes, multiply swaps.

Lesson: The chain rule on a computational graph

Several paths Exam favourite

$$\partial L/\partial x = \Sigma_{\text{paths}} \Pi \text{ local slopes}$$

Gradients add where a value branches.

Lesson: The chain rule on a computational graph

Reverse mode cost Exam favourite

$$\text{cost}(\nabla L) \le c \cdot \text{cost}(L), \; c \text{ a small constant}$$

The cheap gradient principle.

Lesson: Automatic differentiation

Accumulation Exam favourite

$$\partial L/\partial v \mathrel{+}= \partial L/\partial u \cdot \partial u/\partial v \text{ for every child } u \text{ of } v$$

Why gradients must be zeroed each step.

Lesson: Automatic differentiation

Dense layer gradients Exam favourite

$$\partial L/\partial W = X^T G, \quad \partial L/\partial \mathbf{b} = \Sigma_{\text{rows}} G, \quad \partial L/\partial X = GW^T$$

$G = \partial L/\partial Z$.

Lesson: Backpropagation with matrices, and gradient checking

Softmax with cross entropy

Exam favourite

$$\partial L/\partial \mathbf{z} = \mathbf{p} - \mathbf{y}$$

Probabilities minus one hot.

Lesson: Backpropagation with matrices, and gradient checking

ReLU backward

Exam favourite

$$\partial L/\partial z = \partial L/\partial h \cdot [z > 0]$$

Blocks the gradient where the unit was off.

Lesson: Backpropagation with matrices, and gradient checking

Gradient check

$$\text{rel} = |a - n| / (|a| + |n|),\ n = (L(\theta + h) - L(\theta - h)) / 2h$$

Use h near 1e−5.

Lesson: Backpropagation with matrices, and gradient checking

Gradient through L layers

Exam favourite

$$\partial L/\partial \mathbf{h}_1 = \Pi_l \left(W_l^T \operatorname{diag}(\varphi'(\mathbf{z}_l))\right) \partial L/\partial \mathbf{h}_L$$

A product of L factors.

Lesson: Vanishing and exploding gradients

Xavier initialisation

Exam favourite

$$\text{Var}(W) = 2 / (n_{in} + n_{out})$$

Glorot and Bengio (2010).

Lesson: Vanishing and exploding gradients

He initialisation

Exam favourite

$$\text{Var}(W) = 2 / n_{in}$$

For ReLU: half the units are off, so double the variance.

Lesson: Vanishing and exploding gradients

Gradient clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c / \|\mathbf{g}\|)$$

Pascanu and colleagues (2013).

Lesson: Vanishing and exploding gradients

Chain rule

Exam favourite

$$dL/dx = (dL/du)(du/dx)$$

One path.

Lesson: The maths, all in one place

Multivariable chain rule

Exam favourite

$$\partial L/\partial x = \Sigma_i \, (\partial L/\partial u_i)(\partial u_i/\partial x)$$

Sum over every child u_i.

Lesson: The maths, all in one place

Sigmoid derivative

Exam favourite

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Reuses the forward value.

Lesson: The maths, all in one place

Dense layer

Exam favourite

$$\partial L/\partial W = X^T G,\ \partial L/\partial X = G W^T$$

Shapes decide the transposes.

Lesson: The maths, all in one place

Softmax cross entropy

$$\partial L/\partial z_i = p_i - y_i$$

Clean and stable.

Lesson: The maths, all in one place

Backward memory

$$O(\text{number of stored activations})$$

Gradient checkpointing trades compute for memory.

Lesson: The maths, all in one place

TABLES

Table: way to get gradients · exact · cost for n inputs, one output · used for

Way to get gradients	Exact	Cost for n inputs, one output	Used for
By hand	Yes, if no mistakes	Human time	Small models, exams
Finite differences	No, approximate	About n + 1 or 2n evaluations	Checking gradients
Symbolic	Yes	Expressions can blow up in size	Computer algebra systems
Forward mode autodiff	Yes	n passes	Few inputs, many outputs
Reverse mode (backprop)	Yes	About 1 forward plus 1 backward pass	Neural networks: many inputs, one loss

Lesson: Automatic differentiation

Table: · finite differences · symbolic · forward mode

	Finite differences	Symbolic	Forward mode	Reverse mode (backprop)
Accuracy	Approximate, rounding errors	Exact	Exact	Exact
Cost for n parameters	About 2n loss evaluations	Can blow up	n passes	About 2 to 3 passes
Memory	Small	Large expressions	Small	Stores all forward values
Best for	Testing gradients	Formulas on paper	Few inputs	Neural networks

Lesson: Ways to compute gradients compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2236	Root Equals Sum of Children	Easy	Compare the root with the sum of its two children	One comparison does it. In a computational graph an add node's value is exactly the sum of its inputs, which is why its gradient passes straight through. Our helper builds the three node tree from a list.
1376	Time Needed to Inform All Employees	Medium	Recursion from the head down, taking the slowest branch	Build the tree of who manages whom. The time for a manager is their inform time plus the largest time among their reports. Passing a quantity along every path of a tree and combining at each node is the shape of both the forward and the backward pass.
1443	Minimum Time to Collect All Apples in a Tree	Medium	Post order: a subtree costs 2 per edge if it holds an apple	Search depth first. After visiting a child, add its cost plus 2 for the edge there and back, but only if the child's subtree has an apple. Computing children first and combining their results on the way back up is exactly the order of the backward pass.
2192	All Ancestors of a Node in a Directed Acyclic Graph	Medium	A node's ancestors are its parents plus their ancestors, memoised	Store each node's parents and compute ancestors recursively with a cache. In a computational graph, a value's ancestors are everything it depends on, and backpropagation sends gradients to exactly those nodes.
329	Longest Increasing Path in a Matrix	Hard	Depth first search with memoisation over a hidden DAG	Moving to a strictly larger neighbour can never loop, so the cells form a directed acyclic graph. The best path from a cell is 1 plus the best from any larger neighbour, cached so each cell is solved once. Reusing stored results for every node of a DAG is what makes backpropagation cost about the same as one forward pass.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

For f = (x y + z) w followed by a sigmoid and a squared error, the backward pass gives ∂L/∂x = 0.0887 and ∂L/∂w = −0.1774. Nudging x and w by 0.00001 in each direction gives the same numbers to four decimals. This numerical check is how libraries test their gradients.

Lesson: The chain rule on a computational graph

IN THE EXAM In about forty lines, the engine gets d(x·x + x)/dx = 7 at x = 3, adding the three contributions from the three uses of x. It then trains a tanh neuron from a loss of 3.7673 to 0.0047 in 50 steps, and the neuron outputs 0.98 and −0.935 for targets +1 and −1.

Lesson: Automatic differentiation

IN THE EXAM The hand written backward pass agrees with finite differences to a relative error of 7.9e−10 for W1 and 5.6e−10 for W2. Training on 90 overlapping points from 3 classes takes the loss from 1.2295 to 0.3874 and accuracy to 0.822.

Lesson: Backpropagation with matrices, and gradient checking

IN THE EXAM With 20 layers, sigmoid with Xavier initialisation shrinks the gradient from 1.8e−1 at the top to 9.0e−14 at layer 1. Tanh with tiny weights (0.01) is even worse, 6.0e−23. ReLU with Xavier shrinks it about 250 times. ReLU with He initialisation keeps it between 1.1 and 3.3 at every layer.

Lesson: Vanishing and exploding gradients

1.3 Training deep networks: optimisers, normalisation and regularisation

Optimisers, schedules, normalisation, regularisation and a practical recipe for training deep networks.

KEY IDEAS

- **Optimisers** SGD $\eta < 2/\lambda_{max}$. Momentum $v = \beta v + g$. RMSprop divides by $\sqrt{s}$. Adam = both plus bias correction.
- **Defaults** AdamW with η about 1e−3 or 3e−4 and decay 0.01 to 0.1. SGD momentum 0.9 for large CNNs.
- **Schedules** Warm up, then cosine or step decay. Batch × k means learning rate × k.
- **Normalisation** BN over the batch per feature, running averages at test. LN per example, for transformers.
- **Regularisation** Weight decay, inverted dropout (divide by p), early stopping on validation, more data and augmentation.
- **Recipe** Check loss ln C at start, overfit one batch, find the rate, then regularise and tune one thing at a time.

FORMULAS

SGD Exam favourite

$$\theta \leftarrow \theta - \eta\, g$$

g is the mini batch gradient.

Lesson: Optimisers: SGD, momentum, RMSprop and Adam

Momentum Exam favourite

$$v \leftarrow \beta v + g, \quad \theta \leftarrow \theta - \eta v$$

β about 0.9.

Lesson: Optimisers: SGD, momentum, RMSprop and Adam

RMSprop Exam favourite

$$s \leftarrow \rho s + (1-\rho) g^2, \quad \theta \leftarrow \theta - \eta g / (\sqrt{s} + \varepsilon)$$

Per weight step size.

Lesson: Optimisers: SGD, momentum, RMSprop and Adam

Adam Exam favourite

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \quad v \leftarrow \beta_2 v + (1-\beta_2) g^2$$

$\beta_1 = 0.9$, $\beta_2 = 0.999$.

Lesson: Optimisers: SGD, momentum, RMSprop and Adam

Adam update Exam favourite

$$\hat{m} = m/(1-\beta_1^t), \quad \hat{v} = v/(1-\beta_2^t), \quad \theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$$

Bias correction for the zero start.

Lesson: Optimisers: SGD, momentum, RMSprop and Adam

Step decay Exam favourite

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

For example $\gamma = 0.1$ every s epochs.

Lesson: Mini batches and learning rate schedules

Cosine decay

Exam favourite

$$\eta_t = \tfrac{1}{2}\eta_0(1 + \cos(\pi t/T))$$

Smooth from η_0 to 0.

Lesson: Mini batches and learning rate schedules

Linear warm up

Exam favourite

$$\eta_t = \eta_0 \cdot (t + 1)/w \text{ for } t < w$$

Then the main schedule.

Lesson: Mini batches and learning rate schedules

Linear scaling rule

$$\text{batch} \times k \Rightarrow \eta \times k$$

Goyal and colleagues (2017).

Lesson: Mini batches and learning rate schedules

Batch norm

Exam favourite

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}, \quad y = \gamma\hat{x} + \beta$$

Per feature over the batch.

Lesson: Batch normalisation and layer normalisation

Running mean

Exam favourite

$$\mu \leftarrow 0.9\mu + 0.1\mu_B$$

Used at test time.

Lesson: Batch normalisation and layer normalisation

Layer norm

Exam favourite

Same formula with μ and σ over one example's features

No batch dependence.

Lesson: Batch normalisation and layer normalisation

L2 penalty

Exam favourite

$$L' = L + (\lambda/2)\|\mathbf{w}\|^2$$

Adds $\lambda\mathbf{w}$ to the gradient.

Lesson: Regularisation: weight decay, dropout and early stopping

Weight decay step

Exam favourite

$$\mathbf{w} \leftarrow \mathbf{w}(1 - \eta\lambda) - \eta g$$

Same as L2 for plain SGD, not for Adam.

Lesson: Regularisation: weight decay, dropout and early stopping

Inverted dropout

Exam favourite

$$\tilde{h} = h \cdot m / p, \; m \sim \text{Bernoulli}(p)$$

Keeps the expected value. No change at test time.

Lesson: Regularisation: weight decay, dropout and early stopping

Early stopping

keep θ at $\text{argmin}_{\text{epoch}}$ validation loss

Patience: how long to wait for improvement.

Lesson: Regularisation: weight decay, dropout and early stopping

Stability limit for gradient descent

Exam favourite

$$\eta < 2 / \lambda_{max}$$

λ_{max} is the largest curvature (Hessian eigenvalue).

Lesson: The maths, all in one place

Condition number

Exam favourite

$$\kappa = \lambda_{max} / \lambda_{min}$$

Large κ means a narrow valley and slow SGD.

Lesson: The maths, all in one place

Momentum terminal speed

Exam favourite

$$v \to g / (1 - \beta)$$

β = 0.9 gives steps 10 times larger on a steady slope.

Lesson: The maths, all in one place

Adam bias correction

Exam favourite

$$E[m_t] \approx (1 - \beta_1^t)\, E[g]$$

Divide by $1 - \beta^t$ to undo it.

Lesson: The maths, all in one place

Gradient noise

$$\text{Var}(g_B) \approx \text{Var}(g_1) / B$$

Bigger batches give cleaner gradients.

Lesson: The maths, all in one place

AdamW

$$\theta \leftarrow \theta - \eta(\hat{m}/(\sqrt{\hat{v}} + \varepsilon) + \lambda\theta)$$

Decay outside the adaptive scaling.

Lesson: The maths, all in one place

TABLES

Table: batch size · gradient noise · updates per epoch · typical effect

Batch size	Gradient noise	Updates per epoch	Typical effect
1 (pure SGD)	Very high	n	Slow on hardware, noisy, needs a small rate
32 to 512	Moderate	n / batch	Good default, noise helps generalisation
Thousands	Low	Few	Fast per epoch on GPUs, needs larger rate and warm up
Full batch	None	1	Plain gradient descent, many epochs needed

Lesson: Mini batches and learning rate schedules

Table: · batch norm · layer norm

	Batch norm	Layer norm
Statistics over	The batch, per feature	The features, per example
Depends on batch size	Yes, poor with tiny batches	No
Train and test differ	Yes, running averages at test	No
Typical home	CNNs	Transformers, RNNs

Lesson: Batch normalisation and layer normalisation

Table: step · what to do · what it catches

Step	What to do	What it catches
1. Check the loss at start	With C classes the first loss should be about ln C	Wrong labels, bad initialisation
2. Overfit one batch	Training loss on 2 to 10 examples should reach almost 0	Bugs in the model or the loss
3. Find the learning rate	Raise it over a short run and pick a value below where loss explodes	Rates that are too small or too large
4. Train the full model	Adam or SGD with momentum, cosine schedule, warm up	Underfitting
5. Regularise	Weight decay, dropout, augmentation, early stopping	Overfitting
6. Tune	Random search over rate, decay and batch size	Leftover performance

Lesson: A practical recipe for training a network

Table: · sgd · sgd with momentum · rmsprop

	SGD	SGD with momentum	RMSprop	Adam or AdamW
Extra memory per weight	None	1 value	1 value	2 values
Sensitive to the learning rate	Very	Quite	Less	Least
Handles narrow valleys	Poorly	Better	Well when aligned with axes	Well
Typical learning rate	0.01 to 0.1	0.01 to 0.1	0.001	0.001 (3e−4 is common)
Common use	Teaching, simple convex models	Large CNNs on images	RNNs, reinforcement learning	Transformers, most new models

Lesson: Optimisers compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2574	Left and Right Sum Differences	Easy	Keep a running sum from the left and get the right sum from the total	Walk once from left to right. The right sum is the total minus the left sum minus the current value. Keeping one running total instead of recomputing everything is how momentum and Adam store their averages.
1870	Minimum Speed to Arrive on Time	Medium	Binary search for the smallest speed that is fast enough	Being on time only gets easier as the speed rises, so binary search on the answer. Every train but the last waits for the next whole hour. Searching for the largest safe value of a setting is how a learning rate range test works.
2064	Minimized Maximum of Products Distributed to Any Store	Medium	Binary search the largest load and count the stores it needs	For a candidate maximum x, product q needs ceil(q / x) stores. Find the smallest x whose total fits in n stores. Splitting work into equal pieces is the same arithmetic as choosing a batch size.
1552	Magnetic Force Between Two Balls	Medium	Binary search the gap and place balls greedily	Sort the positions. For a candidate gap, place a ball whenever the distance from the last one is big enough, and check whether m balls fit. Keep the largest gap that works.
410	Split Array Largest Sum	Hard	Binary search the largest piece sum and split greedily	For a limit, cut a new piece whenever adding the next number would pass it, and count the pieces. The smallest limit that needs at most k pieces is the answer. This is how data and pipeline parallel training balance work across devices.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

On the valley $f = \frac{1}{2}(x^2 + 100y^2)$, plain SGD is stable only below learning rate $2/100 = 0.02$. At its best safe rate 0.019 it reaches a loss of 0.80 after 60 steps, and at 0.021 it explodes to about 10 million. Momentum reaches 0.105 with a rate of only 0.01, Adam reaches 0.103 and RMSprop reaches 0.00005. RMSprop wins so clearly here because this valley lines up with the axes, so scaling each coordinate separately fixes the problem completely. A tilted valley would be harder for it.

Lesson: Optimisers: SGD, momentum, RMSprop and Adam

IN THE EXAM

Mini batch SGD with batch 8 on a noisy linear regression ends 0.0167 above the best possible loss with a constant learning rate. Step decay, cosine decay and warm up with cosine all end about 0.0002 above it, roughly 80 times closer, with the same number of steps. For batch size, batch 1 makes 6,000 noisy updates and ends 0.177 above the best, batch 64 makes 90 updates and ends 0.0007 above, and the full batch makes only 6 updates in 6 epochs and is still 2.56 above.

Lesson: Mini batches and learning rate schedules

IN THE EXAM

In a 10 layer tanh network with weights a little too small, the activation spread falls from 0.415 at layer 1 to 0.0007 at layer 10. With batch normalisation before each tanh, it stays at about 0.63 in every layer. After BN with $\gamma = 1$ and $\beta = 0$ each feature has mean 0 and standard deviation 1, and $\gamma = 2$ or $\beta = 10$ rescale or shift it exactly as asked.

Lesson: Batch normalisation and layer normalisation

IN THE EXAM

A network with two layers of 64 tanh units, trained on 16 noisy points of sin(x), reaches a training error of 0.0026 but a validation error of 0.596. Weight decay 0.01 gives a validation error of 0.111, dropout with keep 0.8 gives 0.132, and early stopping halts at epoch 447 with 0.118. The noise alone gives about 0.09, so these are close to the best possible.

Lesson: Regularisation: weight decay, dropout and early stopping

STEP 2 Architectures

2.1 Convolutional neural networks

Convolution, pooling, receptive fields, a hand built CNN, ResNet and transfer learning.

KEY IDEAS

- **Convolution** Slide a k × k filter, multiply and add. Weight sharing, local connections. Output $\lfloor (n + 2p - k)/s \rfloor + 1$.
- **Weights** $(k^2 C_{in} + 1) C_{out}$ per conv layer, independent of image size. 1 × 1 conv mixes channels.
- **Pooling** Max or average over windows, halves the size, no weights. Gradient goes to the arg max.
- **Receptive field** $r \leftarrow r + (k - 1)dj$, $j \leftarrow js$. Three 3 × 3 = one 7 × 7 with fewer weights.
- **Architectures** LeNet, AlexNet (ReLU, GPUs), VGG (3 × 3), GoogLeNet (Inception), ResNet (h + F(h)), EfficientNet, ViT.
- **Practice** Pretrained backbone, new head, augmentation that keeps labels true, then fine tune with a smaller learning rate.

FORMULAS

Convolution (as used in deep learning) Exam favourite

$$Y[i, j] = \Sigma_{a, b} X[i \cdot s + a, j \cdot s + b] \cdot K[a, b] + b$$

Strictly a cross correlation: the filter is not flipped.

Lesson: Convolution: a small filter slid over the image

Output size Exam favourite

$$o = \lfloor (n + 2p - k) / s \rfloor + 1$$

Same for height and width.

Lesson: Convolution: a small filter slid over the image

Weights in a conv layer Exam favourite

$$k \times k \times C_{in} \times C_{out} + C_{out}$$

Independent of the image size.

Lesson: Convolution: a small filter slid over the image

Same padding

$$p = (k - 1) / 2 \text{ for stride } 1$$

Keeps the size.

Lesson: Convolution: a small filter slid over the image

Receptive field Exam favourite

$$r_l = r_{l-1} + (k_l - 1) \cdot d_l \cdot j_{l-1}$$

d is the dilation, j the jump.

Lesson: Channels, pooling and the receptive field

Jump Exam favourite

$$j_l = j_{l-1} \cdot s_l$$

Strides multiply.

Lesson: Channels, pooling and the receptive field

Max pooling Exam favourite

$$Y[i, j] = \max \text{ over the window of } X$$

No weights to learn.

Lesson: Channels, pooling and the receptive field

Global average pooling

$$y_c = \text{mean over all positions of channel } c$$

Replaces big dense layers at the end.

Lesson: Channels, pooling and the receptive field

Filter gradient Exam favourite

$$\partial L / \partial K[a, b] = \Sigma_{i, j} \partial L / \partial Y[i, j] \cdot X[i + a, j + b]$$

Summed over every position.

Lesson: Training a CNN on handwritten digits

Max pool gradient Exam favourite

$$\text{Goes to the arg max of each window, 0 elsewhere}$$

Like a switch.

Lesson: Training a CNN on handwritten digits

Residual block Exam favourite

$$\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l, W_l)$$

He and colleagues (2016).

Lesson: Famous architectures and residual connections

Gradient through a residual block Exam favourite

$$\partial L/\partial \mathbf{h}_l = \partial L/\partial \mathbf{h}_{l+1}(I + \partial F/\partial \mathbf{h}_l)$$

The I keeps the gradient alive.

Lesson: Famous architectures and residual connections

1 × 1 convolution

Mixes channels at each pixel: $C_{in} \times C_{out}$ weights

Used to shrink channels cheaply.

Lesson: Famous architectures and residual connections

Output size Exam favourite

$$o = \lfloor (n + 2p - k)/s \rfloor + 1$$

Per spatial dimension.

Lesson: The maths, all in one place

Conv weights Exam favourite

$$(k^2 C_{in} + 1)\, C_{out}$$

Bias per filter.

Lesson: The maths, all in one place

Conv multiply adds Exam favourite

$$o^2 \cdot k^2 \cdot C_{in} \cdot C_{out}$$

Where most compute goes.

Lesson: The maths, all in one place

Receptive field Exam favourite

$$r \leftarrow r + (k - 1)\, d\, j, \quad j \leftarrow j\, s$$

Start r = j = 1.

Lesson: The maths, all in one place

Equivariance

$$\text{conv}(\text{shift}(X)) = \text{shift}(\text{conv}(X))$$

Translation equivariance of convolution.

Lesson: The maths, all in one place

Depthwise separable conv

$k^2 C_{in} + C_{in} C_{out}$ weights instead of $k^2 C_{in} C_{out}$

MobileNet.

Lesson: The maths, all in one place

TABLES

Table: layer · output shape · weights

Layer	Output shape	Weights
Input	8 × 8 × 1	0
Conv 3 × 3, 8 filters, padding 1	8 × 8 × 8	9 × 8 + 8 = 80
ReLU	8 × 8 × 8	0
Max pool 2 × 2	4 × 4 × 8	0
Dense to 10 classes	10	128 × 10 + 10 = 1,290

Lesson: Training a CNN on handwritten digits

Table: network · year · key idea · depth

Network	Year	Key idea	Depth	ImageNet top 5 error
LeNet 5	1998	Conv and pool for digits	7	Not on ImageNet
AlexNet	2012	ReLU, dropout, GPUs	8	15.3%
VGG 16	2014	Only 3 × 3 convolutions	16	7.3%
GoogLeNet	2014	Inception: parallel filter sizes	22	6.7%
ResNet 152	2015	Residual connections	152	3.6%
EfficientNet	2019	Balanced scaling of depth, width, resolution	Varies	About 2.9%
Vision transformer	2020	Attention on image patches	12 to 32	Similar or better at scale

Lesson: Famous architectures and residual connections

Table: your data · similar to imagenet · different from imagenet

Your data	Similar to ImageNet	Different from ImageNet
Small	Freeze the backbone, train a new head	Freeze early layers, train later ones and the head
Large	Fine tune the whole network	Fine tune everything, or train from scratch

Lesson: Transfer learning and data augmentation

Table: · dense network (mlp) · convolutional network · vision transformer

	Dense network (MLP)	Convolutional network	Vision transformer
Connections	Every pixel to every unit	Local window, shared weights	Every patch attends to every patch
Built in assumptions	None	Locality and translation equivariance	Very few
Weights for a 224 × 224 image	Huge	Small, independent of image size	Moderate
Data needed	A lot, and still poor	Works well with modest data	Needs large data or pretraining
Best for	Small tabular inputs	Most vision tasks, phones and edge devices	Large scale vision, multimodal models

Lesson: CNN, dense network and vision transformer compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
661	Image Smoother	Easy	Average each cell with its neighbours that exist	For every cell, add up the values in the 3 × 3 window that fall inside the image and divide by how many there were. This is exactly a blur filter, the first filter in the animation, with the edges handled by counting instead of padding.
835	Image Overlap	Medium	Count, for every shift, how many 1s line up	Pair every 1 in the first image with every 1 in the second and record the shift between them. The shift seen most often gives the largest overlap. Sliding one image over another and counting matches is a cross correlation, the same operation a convolution layer computes.
304	Range Sum Query 2D, Immutable	Medium	Precompute a 2D prefix sum (an integral image)	Store s[i][j], the sum of everything above and to the left. Any rectangle sum is then four lookups. Integral images make box filters cost the same for any window size, and they powered the classic Viola and Jones face detector. Our helper answers a list of queries.
221	Maximal Square	Medium	The square ending at a cell is 1 plus the smallest of its three neighbours	dp[i][j] is the side of the largest all 1 square whose bottom right corner is (i, j). It equals 1 plus the minimum of the cells above, to the left and diagonally up left. Each cell depends on a small window of earlier cells, the same local pattern a convolution uses.
85	Maximal Rectangle	Hard	Turn each row into a histogram and find its largest rectangle with a stack	Keep, for each column, how many 1s stand above the current row. Each row is then a histogram, and a monotonic stack finds its largest rectangle in one pass. Building up a 2D answer row by row from 1D summaries is how many image algorithms stay fast.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

On an image that is dark on the left and bright on the right, the vertical edge filter gives 27 exactly where the brightness jumps and 0 elsewhere, and the horizontal edge filter gives 0 everywhere. A 224 × 224 image with a 7 × 7 filter, padding 3 and stride 2 gives a 112 × 112 output, the first layer of ResNet.

Lesson: Convolution: a small filter slid over the image

IN THE EXAM

2 × 2 max pooling turns the 4 × 4 map into [[6, 5], [7, 9]]. A 3 × 3 convolution with 16 filters on a 32 × 32 × 3 image has 448 weights, while a dense layer producing the same 32 × 32 × 16 output would need about 50.3 million, 112,384 times more. Through conv, conv, pool, conv, pool, conv the receptive field grows 3, 5, 6, 10, 12, 20.

Lesson: Channels, pooling and the receptive field

IN THE EXAM

On the 8 × 8 scikit learn digits, trained with SGD with momentum for 8 epochs, test accuracy rises from 0.88 after the first epoch to about 0.97. The network has only 1,370 weights: 80 in the 8 filters and 1,290 in the final dense layer.

Lesson: Training a CNN on handwritten digits

IN THE EXAM

With 50 layers and small weights, the gradient that reaches layer 1 of a plain network is 2.7e−50 times the gradient at the output: it has vanished. With the same weights and residual connections it is about 10.5 times the output gradient, because the skip path carries it straight through. Real ResNets also use batch normalisation to keep the sizes steady.

Lesson: Famous architectures and residual connections

2.2 Recurrent networks, LSTM and GRU

RNNs and BPTT, the long term dependency problem, LSTM and GRU gates, and sequence model variants.

KEY IDEAS

- **RNN** $h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b)$, shared weights, any length. Train with BPTT.
- **Long term problem** Gradient across k steps is a product of k Jacobians: vanishes or explodes. Clip for explosions.
- **LSTM** $c = f \odot c + i \odot g$, $h = o \odot \tanh(c)$. Cell path multiplied only by f. Positive forget bias.
- **GRU** $h = (1 - z)h + z\,\tilde{h}$, reset gate r. 3 blocks against 4 for LSTM.
- **Weights** blocks $\times (n_{in}H + H^2 + H)$: RNN 1, GRU 3, LSTM 4.
- **Variants** Bidirectional needs the whole sequence. Encoder decoder with teacher forcing. Attention fixes the bottleneck.

FORMULAS

RNN cell — Exam favourite

$$\mathbf{h}_t = \tanh(\mathbf{x}_t W_{xh} + \mathbf{h}_{t-1} W_{hh} + \mathbf{b})$$

Same weights at every t.

Lesson: Recurrent networks: one cell reused at every time step

Output — Exam favourite

$$\mathbf{y}_t = \mathbf{h}_t W_{hy} + \mathbf{c}$$

At every step or only the last.

Lesson: Recurrent networks: one cell reused at every time step

BPTT — Exam favourite

$$\partial L / \partial W_{hh} = \Sigma_t \, \partial L_t / \partial W_{hh}$$

Sum over the unrolled copies.

Lesson: Recurrent networks: one cell reused at every time step

Jacobian back one step

$$\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} = \mathrm{diag}(1 - \mathbf{h}_t^2)\, W_{hh}^T$$

Multiplied once per step back.

Lesson: Recurrent networks: one cell reused at every time step

Gradient across k steps — Exam favourite

$$\partial \mathbf{h}_t / \partial \mathbf{h}_{t-k} = \Pi_{j=0}^{k-1} \mathrm{diag}(1 - \mathbf{h}_{t-j}^2)\, W_{hh}^T$$

A product of k matrices.

Lesson: The long term dependency problem

Rough size — Exam favourite

$$\approx (\gamma \cdot \sigma_{max}(W_{hh}))^k$$

$\gamma \le 1$ is the tanh slope. Below 1 vanishes, above 1 can explode.

Lesson: The long term dependency problem

Gradient clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$$

Pascanu and colleagues (2013).

Lesson: The long term dependency problem

Gates — Exam favourite

$$\mathbf{i}, \mathbf{f}, \mathbf{o} = \sigma(\mathbf{x}_t W + \mathbf{h}_{t-1} U + \mathbf{b})$$

Each has its own W, U, b.

Lesson: LSTM: a memory cell with gates

Candidate — Exam favourite

$$\mathbf{g} = \tanh(\mathbf{x}_t W_g + \mathbf{h}_{t-1} U_g + \mathbf{b}_g)$$

The new information.

Lesson: LSTM: a memory cell with gates

Cell update — Exam favourite

$$\mathbf{c}_t = \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{i} \odot \mathbf{g}$$

Keep some, add some.

Lesson: LSTM: a memory cell with gates

Hidden state Exam favourite

$$\mathbf{h}_t = \mathbf{o} \odot \tanh(\mathbf{c}_t)$$

What the cell shows.

Lesson: LSTM: a memory cell with gates

Gradient along the cell Exam favourite

$$\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f})$$

No weight matrix, no tanh in the way.

Lesson: LSTM: a memory cell with gates

GRU update Exam favourite

$$\mathbf{h}_t = (1 - \mathbf{z}) \odot \mathbf{h}_{t-1} + \mathbf{z} \odot \tilde{\mathbf{h}}_t$$

z near 0 copies the old state.

Lesson: GRU, bidirectional and sequence to sequence models

GRU candidate Exam favourite

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{x}_t \mathbf{W} + (\mathbf{r} \odot \mathbf{h}_{t-1})\mathbf{U} + \mathbf{b})$$

r near 0 ignores the past.

Lesson: GRU, bidirectional and sequence to sequence models

Weights per layer Exam favourite

$$\text{gates} \times (n_{in} H + H^2 + H)$$

1 for RNN, 3 for GRU, 4 for LSTM.

Lesson: GRU, bidirectional and sequence to sequence models

Bidirectional state

$$\mathbf{h}_t = [\mathbf{h}_{\rightarrow t}, \mathbf{h}_{\leftarrow t}]$$

Concatenate both directions.

Lesson: GRU, bidirectional and sequence to sequence models

RNN Exam favourite

$$\mathbf{h}_t = \varphi(\mathbf{x}_t \mathbf{W}_{xh} + \mathbf{h}_{t-1} \mathbf{W}_{hh} + \mathbf{b})$$

φ usually tanh.

Lesson: The maths, all in one place

LSTM cell Exam favourite

$$\mathbf{c}_t = \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{i} \odot \mathbf{g}, \quad \mathbf{h}_t = \mathbf{o} \odot \tanh(\mathbf{c}_t)$$

Additive memory.

Lesson: The maths, all in one place

GRU Exam favourite

$$\mathbf{h}_t = (1 - \mathbf{z})\mathbf{h}_{t-1} + \mathbf{z}\tilde{\mathbf{h}}_t$$

Interpolation.

Lesson: The maths, all in one place

Memory after k steps with forget value f Exam favourite

$$c_{t+k} \approx f^k c_t$$

f = 0.99 keeps 90% after 10 steps.

Lesson: The maths, all in one place

Forget bias for a target f

$$b = \ln(f / (1 - f))$$

The inverse sigmoid (logit).

Lesson: The maths, all in one place

Sequence loss

$$L = \Sigma_t -\ln p(y_t \mid y_{<t}, \mathbf{x})$$

Cross entropy at every step.

Lesson: The maths, all in one place

TABLES

Table: model · shape · typical use

Model	Shape	Typical use
Many to one	Read a sequence, output one label	Sentiment, activity recognition
Many to many, aligned	One output per input step	Part of speech tagging, frame labelling
One to many	One input, generate a sequence	Image captioning, music generation
Encoder decoder	Read all, then generate	Translation, summarisation, speech to text

Lesson: GRU, bidirectional and sequence to sequence models

Table: · plain rnn · lstm · gru

	Plain RNN	LSTM	GRU	Transformer
Memory	Short, about 10 to 20 steps	Long, hundreds of steps	Long, similar to LSTM	Whole context window directly
Weights per layer	$1 \times (n_{in}H + H^2 + H)$	$4 \times$	$3 \times$	Much larger, attention plus feed forward
Training speed	Sequential in time	Sequential in time	Sequential in time	Parallel over positions
Cost per step at inference	Constant	Constant	Constant	Grows with context length
Common today in	Teaching	Speech, time series, small devices	Same as LSTM, smaller	Language, vision, almost everything large

Lesson: RNN, LSTM, GRU and transformer compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
20	Valid Parentheses	Easy	Push opening brackets and pop on each closing one	Read the string one character at a time, keeping a stack of brackets still open. A closing bracket must match the top of the stack. Like an RNN, the stack is a state that carries what matters from earlier in the sequence.
901	Online Stock Span	Medium	Keep a stack of (price, span) and merge smaller prices into the new one	Prices arrive one at a time, and the answer must come immediately, exactly the streaming setting of an RNN. Pop every earlier price that is not higher and add its span, so each price is pushed and popped once. Our helper feeds a list of prices.
394	Decode String	Medium	Save the current text and count on a stack at each opening bracket	Build the current piece character by character. At "[" save it with the repeat count, at "]" pop and repeat. Nested structure in a sequence is exactly what a single fixed size state finds hard, and why a stack makes it easy.
1143	Longest Common Subsequence	Medium	A table over prefixes of both strings	dp[i][j] is the answer for the first i characters of a and the first j of b. Matching characters extend the diagonal, otherwise take the better of dropping one character. Aligning two sequences is the problem encoder decoder models and attention solve for translation.
72	Edit Distance	Hard	Each cell is the cheapest of insert, delete or replace from a neighbour	dp[i][j] is the number of edits to turn the first i letters of word1 into the first j letters of word2. Speech recognition is scored with this distance at the word level (the word error rate), and CTC training for speech RNNs uses a closely related table.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

A 4 unit RNN reads 5 steps and then 100 steps, and its state is still just 4 numbers. Backpropagating from step 100, the sensitivity of the final state to earlier states falls from 1.39 one step back to 0.61 ten steps back, 0.0088 twenty steps back and 2.0e−7 fifty steps back. We checked the value ten steps back against finite differences.

Lesson: Recurrent networks: one cell reused at every time step

IN THE EXAM

The task: see a number at step 1 with a marker, then noise as large as the signal, and output the number at the end. Always answering 0 scores 0.333. A 16 unit plain RNN trained with BPTT and clipping reaches a test error of 0.0003 at 5 steps and 0.0007 at 20 steps, but 0.328 at 40 steps and 0.345 at 60 steps: no better than guessing 0.

Lesson: The long term dependency problem

IN THE EXAM

On the same memory task, an LSTM with the usual forget bias of 1 learns 20 steps (error 0.0006) but fails at 60 steps (0.345). With forget bias 5, so that f starts near 0.99, it learns 60 steps with an error of 0.0010, where the plain RNN failed already at 40. The hand written LSTM gradients were checked against finite differences.

Lesson: LSTM: a memory cell with gates

IN THE EXAM

With the update gate bias set to −6, a GRU copies its state almost unchanged through 50 noisy steps: (0.5, −0.5, 0.2, 0.9) becomes (0.37, −0.49, 0.09, 0.81). For 100 inputs and 128 hidden units, an RNN layer has 29,312 weights, a GRU 87,936 and an LSTM 117,248.

Lesson: GRU, bidirectional and sequence to sequence models

2.3 Attention and transformers

Attention, multi head attention, the transformer block, positional encoding, BERT, GPT and decoding.

KEY IDEAS

- **Attention** softmax($QK^T/\sqrt{d_k}$)V. Q, K, V are learned maps of the input. Weights sum to 1.
- **Masking** Causal mask for decoders (GPT). Padding masks hide padding tokens.
- **Block** MHA + FFN (4d hidden), each with residual and layer norm. About $12d^2$ weights. $d_{head} = d/h$.
- **Positions** Sinusoidal, learned or rotary encodings. Attention alone is order blind.
- **Why it won** Path length 1, parallel over positions. Cost $O(T^2d)$, helped by FlashAttention and KV caches.
- **Models and decoding** BERT encoder (masked), GPT decoder (next token), T5 encoder decoder, ViT patches. Temperature, top k, top p, perplexity.

FORMULAS

Scaled dot product attention Exam favourite

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

Vaswani and colleagues (2017).

Lesson: Attention: every word looks at every other word

Projections Exam favourite

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

All learned.

Lesson: Attention: every word looks at every other word

Causal mask Exam favourite

$$\text{score}_{ij} = -\infty \text{ for } j > i$$

Decoders cannot peek ahead.

Lesson: Attention: every word looks at every other word

Why $\sqrt{d_k}$

$$\text{Var}(q \cdot k) = d_k \text{ for unit variance entries}$$

Keeps softmax away from saturation.

Lesson: Attention: every word looks at every other word

Multi head attention Exam favourite

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$.

Lesson: Multi head attention and the transformer block

Transformer block (post norm) Exam favourite

$$\mathbf{z} = \text{LN}(\mathbf{x} + \text{MHA}(\mathbf{x})), \quad \mathbf{y} = \text{LN}(\mathbf{z} + \text{FFN}(\mathbf{z}))$$

Pre norm uses x + MHA(LN(x)).

Lesson: Multi head attention and the transformer block

Feed forward network Exam favourite

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2$$

Usually 4 d_{model} hidden units, often GELU.

Lesson: Multi head attention and the transformer block

Sinusoidal encoding Exam favourite

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d}), \quad PE(pos, 2i+1) = \cos(\ldots)$$

Wavelengths from 2π to about 10000 · 2π.

Lesson: Multi head attention and the transformer block

Weights per block

$$\approx 4d^2 \text{ (attention)} + 8d^2 \text{ (FFN)} = 12d^2$$

Ignoring biases and layer norm.

Lesson: Multi head attention and the transformer block

Language model Exam favourite

$$p(\mathbf{x}) = \Pi_t \, p(x_t \mid x_{<t})$$

Chain rule of probability.

Lesson: BERT, GPT and how text is generated

Temperature Exam favourite

$$p_i = \text{softmax}(z_i / T)$$

T < 1 sharper, T > 1 flatter.

Lesson: BERT, GPT and how text is generated

Perplexity Exam favourite

$$PPL = \exp(-(1/N) \, \Sigma \ln p(x_t \mid x_{<t}))$$

Lower is better.

Lesson: BERT, GPT and how text is generated

Top p (nucleus)

Smallest set whose probabilities sum to at least p

Holtzman and colleagues (2020).

Lesson: BERT, GPT and how text is generated

Attention Exam favourite

$$\text{softmax}(QK^T/\sqrt{d_k} + M)V$$

M = 0 or −∞ for masking.

Lesson: The maths, all in one place

Shapes Exam favourite

$$Q: T \times d_k,\ K: T \times d_k,\ V: T \times d_v \Rightarrow \text{output } T \times d_v$$

Scores T × T.

Lesson: The maths, all in one place

Heads Exam favourite

$$d_{head} = d_{model} / h$$

Same total cost as one big head.

Lesson: The maths, all in one place

Cost Exam favourite

$$O(T^2 d) \text{ time, } O(T^2) \text{ memory for the scores}$$

Per layer.

Lesson: The maths, all in one place

KV cache

$$2 \times \text{layers} \times T \times d_{model} \text{ numbers}$$

Stored during generation.

Lesson: The maths, all in one place

Cross entropy and perplexity

$$PPL = e^H, \text{ H in nats per token}$$

H = ln 4 gives PPL 4.

Lesson: The maths, all in one place

TABLES

Table: layer type · cost per layer · sequential steps · longest path between positions

Layer type	Cost per layer	Sequential steps	Longest path between positions
Self attention	$O(T^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(T d^2)$	$O(T)$	$O(T)$
Convolution, width k	$O(k T d^2)$	$O(1)$	$O(\log_k T)$

Lesson: Why attention beats recurrence on long distances

Table: · bert style (encoder) · gpt style (decoder) · t5 style (encoder decoder)

	BERT style (encoder)	GPT style (decoder)	T5 style (encoder decoder)
Attention	Both directions	Causal, left to right	Encoder both ways, decoder causal
Pretraining task	Masked tokens, fill the blank	Next token	Fill in masked spans
Good at	Classification, tagging, search	Generation, chat, code	Translation, summarisation
Examples	BERT, RoBERTa	GPT family, Llama, Mistral	T5, BART

Lesson: BERT, GPT and how text is generated

Table: · encoder (bert) · decoder (gpt) · vision transformer (vit)

	Encoder (BERT)	Decoder (GPT)	Vision transformer (ViT)	Encoder decoder (T5)
Input	Tokens	Tokens	Image patches of 16 × 16 pixels	Tokens
Mask	None	Causal	None	Causal in the decoder
Output	One vector per token	Next token probabilities	Class or features	A new sequence
Typical use	Search, classification	Chat, code, reasoning	Image classification, multimodal	Translation, summarisation

Lesson: Transformer families compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
706	Design HashMap	Easy	Buckets of key value pairs chosen by key modulo the size	Hash each key to a bucket and keep a small list of pairs there. A hash map is hard attention: a query matches exactly one key and returns its value. Self attention is the soft version, matching every key a little and blending the values. Our helper runs a list of operations.
208	Implement Trie (Prefix Tree)	Medium	Nested dictionaries, one level per character, with an end marker	Walk one character at a time, creating children as needed. Tokenisers for transformers, such as byte pair encoding, look up the longest known piece of text quickly with exactly this structure. Our helper inserts words and answers queries.
981	Time Based Key-Value Store	Medium	Keep timestamps sorted per key and binary search the latest one not after t	Store each key's values in time order and use bisect to find the last one at or before the query time. Only looking at entries up to the current time is the idea behind the causal mask in a GPT style decoder.
215	Kth Largest Element in an Array	Medium	A min heap of size k keeps the k largest values seen so far	Push each value and pop the smallest whenever the heap grows past k. The top of the heap is the answer. Top k sampling does this every time a language model picks a token: keep the k most likely tokens and sample among them.
23	Merge k Sorted Lists	Hard	A heap holds the current head of every list	Put the first node of each list in a min heap. Pop the smallest, attach it, and push its successor. Beam search for text generation keeps a similar heap of the best partial sequences and expands the most promising ones. Our helper builds linked lists from Python lists.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

With 4 hand picked tokens, the first query puts weight 0.539 on the key that points the same way and 0.065 on the one that points away. A query of all zeros gives equal weights of 0.25, so it simply averages the values. With the causal mask, the first token can only attend to itself (weight 1). For random vectors, the spread of q · k grows from 2.0 at d = 4 to 22.6 at d = 512, and dividing by √d brings it back to 1.

Lesson: Attention: every word looks at every other word

IN THE EXAM A block maps 6 vectors of size 16 to 6 vectors of size 16, and each head's attention rows sum to 1. Without positional encodings, reversing the input tokens only reverses the outputs, which shows that attention alone cannot see order. One block with $d_{model} = 768$ has about 7.1 million weights, so BERT base's 12 blocks hold 85 million. GPT 3's 96 blocks of size 12,288 hold about 174 billion.

Lesson: Multi head attention and the transformer block

IN THE EXAM On the exact memory task where the plain RNN failed from 40 steps and the LSTM needed a forget bias of 5 to manage 60, a single attention head learns 20, 60 and even 200 steps in 600 training steps. Its test error is below 1e−5, and it puts 0.987, 0.996 and 1.000 of its attention on the marked token.

Lesson: Why attention beats recurrence on long distances

IN THE EXAM For "The cat sat on the", temperature 0.5 gives "mat" a probability of 0.867, temperature 2 lowers it to 0.456 and spreads the rest. Top k = 2 keeps mat and sofa. Top p = 0.9 keeps mat, sofa and roof, which hold 0.984 of the probability. True next words with probabilities 0.60, 0.25, 0.10 and 0.40 give a perplexity of 3.59. Going from 2,048 to 8,192 tokens multiplies the attention scores per head by 16.

Lesson: BERT, GPT and how text is generated

STEP 3 Generative models

3.1 Autoencoders and variational autoencoders

Autoencoders, denoising and anomaly detection, VAEs with the ELBO, and discrete latent codes.

KEY IDEAS

- **Autoencoder** Encoder to a small code, decoder back. Train on reconstruction error, no labels.
- **Versus PCA** Linear autoencoder = PCA subspace. Nonlinear layers rebuild curved data better.
- **Uses** Denoising (noisy in, clean out). Anomalies: train on normal data, flag high reconstruction error.
- **VAE** Encoder gives μ, log σ². z = μ + σε. Loss = reconstruction + KL = negative ELBO.
- **KL** ½Σ(μ² + σ² − 1 − ln σ²), zero only for N(0, 1). β weights it.
- **Discrete and latent** VQ-VAE: nearest codebook index, $\log_2$(size) bits. Latent spaces make diffusion cheap.

FORMULAS

Autoencoder Exam favourite

$$\mathbf{z} = f_{enc}(\mathbf{x}), \quad \hat{\mathbf{x}} = f_{dec}(\mathbf{z})$$

dim(z) much smaller than dim(x).

Lesson: Autoencoders: compress, then rebuild

Reconstruction loss Exam favourite

$L = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$, or binary cross entropy for values in $[0, 1]$

No labels needed.

Lesson: Autoencoders: compress, then rebuild

Linear autoencoder Exam favourite

Optimal solution spans the top k principal components

Baldi and Hornik (1989).

Lesson: Autoencoders: compress, then rebuild

Reparameterisation Exam favourite

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim N(0, I)$$

Randomness moved outside the network.

Lesson: Variational autoencoders

ELBO Exam favourite

$$\ln p(\mathbf{x}) \geq E_{q(z|x)}[\ln p(\mathbf{x}|z)] - KL(q(z|\mathbf{x}) \,||\, p(z))$$

Maximise the lower bound.

Lesson: Variational autoencoders

Gaussian KL Exam favourite

$$KL = \tfrac{1}{2} \Sigma_j (\mu_j^2 + \sigma_j^2 - 1 - \ln \sigma_j^2)$$

Against N(0, I).

Lesson: Variational autoencoders

β VAE

$$L = \text{reconstruction} + \beta \cdot KL$$

Higgins and colleagues (2017).

Lesson: Variational autoencoders

Quantisation Exam favourite

$$z_q = e_k, \quad k = \text{argmin}_j \, ||z_e - e_j||$$

Nearest codebook vector.

Lesson: Discrete codes and latent spaces in modern models

Bits per token Exam favourite

$$\log_2(\text{codebook size})$$

256 entries = 8 bits.

Lesson: Discrete codes and latent spaces in modern models

VQ-VAE loss

$$||\mathbf{x} - \hat{\mathbf{x}}||^2 + ||\text{sg}[z_e] - e||^2 + \beta||z_e - \text{sg}[e]||^2$$

sg = stop gradient.

Lesson: Discrete codes and latent spaces in modern models

Reconstruction (Gaussian) Exam favourite

$$-\ln p(\mathbf{x}|z) \propto ||\mathbf{x} - \hat{\mathbf{x}}||^2$$

Squared error.

Lesson: The maths, all in one place

Reconstruction (Bernoulli) Exam favourite

$$-\Sigma \, [x \ln \hat{x} + (1 - x) \ln(1 - \hat{x})]$$

Binary cross entropy.

Lesson: The maths, all in one place

KL of one dimension Exam favourite

$$\tfrac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$$

Zero only at $\mu = 0$, $\sigma = 1$.

Lesson: The maths, all in one place

Log variance Exam favourite

$$\sigma = \exp(\tfrac{1}{2} \log \sigma^2)$$

Network outputs $\log \sigma^2$ for stability.

Lesson: The maths, all in one place

ELBO gap

$$\ln p(\mathbf{x}) - \text{ELBO} = KL(q(z|\mathbf{x}) \,||\, p(z|\mathbf{x}))$$

Tight when q matches the true posterior.

Lesson: The maths, all in one place

TABLES

Table: variant · what changes · used for

Variant	What changes	Used for
Undercomplete	Small bottleneck	Compression, features
Denoising	Corrupt the input, rebuild the clean one	Cleaning data, robust features
Sparse	Penalise active code units	Interpretable features
Contractive	Penalise sensitivity of the code to the input	Robust features
Variational (VAE)	Code is a distribution, KL penalty	Generating new data

Lesson: Denoising and anomaly detection

Table: · pca · autoencoder · variational autoencoder

	PCA	Autoencoder	Variational autoencoder	VQ-VAE
Mapping	Linear	Nonlinear	Nonlinear, probabilistic	Nonlinear, discrete
Code	Fixed point	Fixed point	A distribution $N(\mu, \sigma^2)$	A codebook index
Can generate new data	Poorly	Poorly, gaps in code space	Yes, sample from the prior	Yes, with a prior over tokens
Training	Closed form (SVD)	Gradient descent	Gradient descent with reparameterisation	Gradient descent with straight through
Typical use	Quick compression, visualising	Denoising, anomaly detection	Generation, smooth latent spaces	Image and audio tokens, latent generators

Lesson: Autoencoder, VAE and PCA compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
461	Hamming Distance	Easy	XOR the numbers and count the 1 bits	Bits that differ become 1 after XOR, so count them. Binary codes compared by Hamming distance are used for fast similarity search with learned hash codes, a close cousin of autoencoder codes.
535	Encode and Decode TinyURL	Medium	Map each long URL to a short code and back with two dictionaries	The encoder turns a long URL into a short code and the decoder turns the code back. This is lossless, unlike an autoencoder, but the same encoder and decoder pair. Our helper checks that every URL survives a round trip.
443	String Compression	Medium	Two pointers: read a run, write the character and its count	Walk through runs of the same character and overwrite the array in place with the character followed by the run length. Run length encoding is a simple lossless compressor, the opposite end from an autoencoder's learned, lossy one. Our helper returns the new length and the compressed text.
89	Gray Code	Medium	i XOR (i shifted right by 1) gives the i-th Gray code	Neighbouring Gray codes differ in exactly one bit, so a small step in the sequence is a small change in the code. A good VAE latent space has the same property: nearby codes decode to similar outputs.
297	Serialize and Deserialize Binary Tree	Hard	Preorder with a marker for empty children, then rebuild recursively	Write the value of each node before its children and "#" for a missing child. Reading the list back in the same order rebuilds the exact tree. An encoder and decoder that must agree on a format is the core of any autoencoder. Our helper builds a tree from level order, serialises it and checks the round trip.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Compressing 8 × 8 digits (64 pixels) to 2 numbers, PCA reaches a test error of 0.0534 and a nonlinear autoencoder 0.0357. With 8 numbers, PCA gives 0.0245 and the autoencoder 0.0147. Just predicting the average image gives 0.0743.

Lesson: Autoencoders: compress, then rebuild

IN THE EXAM

Adding noise with standard deviation 0.3 gives the test digits an error of 0.0524, and the denoising network brings it down to 0.0324. An autoencoder trained only on digits 0 to 4 rebuilds them with error 0.0322 but digits 5 to 9 with 0.0605, and using the error as an anomaly score separates the two groups with a ROC AUC of 0.901.

Lesson: Denoising and anomaly detection

IN THE EXAM

A VAE with a 2 number code trained on the digits for 200 epochs lowers its loss (negative ELBO) from 36.3 to 24.4 nats per image: reconstruction 21.75 plus KL 2.62. Different digits get different average codes, for example (−1.33, −0.39) for 1 and (1.33, 0.92) for 0. Decoding the point halfway between them gives a shape in between, a smooth path through the code space.

Lesson: Variational autoencoders

IN THE EXAM

Snapping 8 number codes of the digits to a codebook of 4 vectors (2 bits per image) gives an error of 0.0549, 16 vectors (4 bits) give 0.0375 and 256 vectors (8 bits) give 0.0273, approaching the continuous code's 0.0239. Each image becomes a single token id. Here the codebook is learned with k means on PCA codes, a simple stand in for a trained VQ-VAE.

Lesson: Discrete codes and latent spaces in modern models

3.2 Generative adversarial networks

The GAN game, its theory and failures, Wasserstein GANs, conditional GANs and StyleGAN, and FID.

KEY IDEAS

- **GAN game** Generator: noise to fake. Discriminator: real or fake. Alternate updates, min over G of max over D.
- **Losses** $L_D = -\ln D(x) - \ln(1 - D(G(z)))$. Generator: $-\ln D(G(z))$ (non saturating).
- **Theory** $D^* = p_{data} / (p_{data} + p_g)$. $V(D^*, G) = -\ln 4 + 2$ JS. Optimum $p_g = p_{data}$, $D = ½$.
- **Failures** Saturation, mode collapse, oscillation, D too strong. Fixes: minibatch features, TTUR, penalties, augmentation.
- **WGAN** Critic outputs real numbers, estimates earth mover distance. Keep 1 Lipschitz: gradient penalty or spectral norm.
- **Variants and FID** DCGAN, conditional, pix2pix, CycleGAN (cycle loss), StyleGAN. FID: feature means and covariances, lower is better.

FORMULAS

Minimax game Exam favourite

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}}[\ln D(x)] + E_{z \sim p_z}[\ln(1 - D(G(z)))]$$

Goodfellow and colleagues (2014).

Lesson: The GAN game: a generator against a discriminator

Discriminator loss Exam favourite

$$L_D = -\ln D(x) - \ln(1 - D(G(z)))$$

Binary cross entropy, real = 1, fake = 0.

Lesson: The GAN game: a generator against a discriminator

Generator loss (non saturating) Exam favourite

$$L_G = -\ln D(G(z))$$

Used in practice.

Lesson: The GAN game: a generator against a discriminator

Optimal discriminator Exam favourite

$$D^*(x) = p_{data}(x) / (p_{data}(x) + p_g(x))$$

For a fixed generator.

Lesson: The best discriminator, the right loss, and mode collapse

Value at the optimum Exam favourite

$$V(D^*, G) = -\ln 4 + 2\,\mathrm{JS}(p_{data} \,\|\, p_g)$$

Smallest, $-\ln 4$, when $p_g = p_{data}$.

Lesson: The best discriminator, the right loss, and mode collapse

Gradient on the logit t Exam favourite

$$\text{saturating: } -D, \quad \text{non saturating: } -(1 - D)$$

$D = \sigma(t)$.

Lesson: The best discriminator, the right loss, and mode collapse

Wasserstein distance Exam favourite

$$W(p, q) = \inf_{\gamma} E_{(x, y)\sim\gamma}\|x - y\|$$

Cheapest way to move p onto q.

Lesson: Wasserstein GANs and other stable losses

Kantorovich Rubinstein duality Exam favourite

$$W(p, q) = \sup_{\|f\|_L \le 1} E_p[f(x)] - E_q[f(x)]$$

What the critic estimates.

Lesson: Wasserstein GANs and other stable losses

Gradient penalty Exam favourite

$$\lambda\, E[(\|\nabla_{\hat{x}} f(\hat{x})\| - 1)^2], \quad \lambda = 10$$

Gulrajani and colleagues (2017).

Lesson: Wasserstein GANs and other stable losses

Least squares GAN

$$L_D = (D(x) - 1)^2 + D(G(z))^2$$

Mao and colleagues (2017).

Lesson: Wasserstein GANs and other stable losses

Fréchet distance of Gaussians Exam favourite

$$FID = \|\mu_r - \mu_g\|^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$$

Lower is better.

Lesson: DCGAN, conditional GANs, StyleGAN, and measuring quality

Cycle consistency Exam favourite

$$L_{cyc} = \|F(G(x)) - x\|_1 + \|G(F(y)) - y\|_1$$

CycleGAN.

Lesson: DCGAN, conditional GANs, StyleGAN, and measuring quality

Transposed convolution size Exam favourite

$$out = (in - 1) \times stride - 2 \times pad + kernel$$

Upsampling in DCGAN.

Lesson: DCGAN, conditional GANs, StyleGAN, and measuring quality

GAN value Exam favourite

$$V(D, G) = E_{data}[\ln D(x)] + E_z[\ln(1 - D(G(z)))]$$

D maximises, G minimises.

Lesson: The maths, all in one place

Optimal D Exam favourite

$$D^*(x) = p_{data} / (p_{data} + p_g)$$

Pointwise maximum.

Lesson: The maths, all in one place

JS divergence Exam favourite

$$JS(p \,\|\, q) = \tfrac{1}{2} KL(p \,\|\, m) + \tfrac{1}{2} KL(q \,\|\, m), \quad m = \tfrac{1}{2}(p + q)$$

Between 0 and ln 2.

Lesson: The maths, all in one place

WGAN critic Exam favourite

$$\max_{f \text{ 1 Lipschitz}} E_{data}[f(x)] - E_z[f(G(z))]$$

Estimates $W(p_{data}, p_g)$.

Lesson: The maths, all in one place

FID

$$\|\mu_r - \mu_g\|^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$$

In one dimension: $(\mu_r - \mu_g)^2 + (\sigma_r - \sigma_g)^2$.

Lesson: The maths, all in one place

TABLES

Table: problem · what you see · common fixes

Problem	What you see	Common fixes
Saturating generator loss	Generator barely changes early on	Non saturating loss –ln D(G(z))
Mode collapse	Many samples look the same	Minibatch features, Wasserstein loss, unrolled GANs
Oscillation	Samples keep changing, losses swing	Lower learning rates, two time scale updates (TTUR), averaging generator weights
Discriminator too strong	D near 100 percent, G stuck	Gradient penalty, spectral normalisation, label smoothing, data augmentation (DiffAugment, ADA)

Lesson: The best discriminator, the right loss, and mode collapse

Table: model · idea · known for

Model	Idea	Known for
DCGAN (2016)	All convolutional generator and discriminator	First stable image GAN recipe
Conditional GAN (2014)	Label given to G and D	Class controlled samples
pix2pix (2017)	Conditional GAN plus an L1 loss on paired images	Sketch to photo, maps to aerial views
CycleGAN (2017)	Two generators and a cycle consistency loss	Horses to zebras without paired data
StyleGAN (2019 to 2021)	Style vector controls each resolution	Photorealistic faces, editing by style
GAN discriminators today	Adversarial loss inside other models	Super resolution, decoders of latent diffusion

Lesson: DCGAN, conditional GANs, StyleGAN, and measuring quality

Table: · gan · vae · diffusion model

	GAN	VAE	Diffusion model	Autoregressive model
How it learns	Generator fools a discriminator	Maximise the ELBO	Learn to remove noise step by step	Predict the next piece
Sample quality	Very sharp	Often blurry	Very sharp	Sharp
Variety (mode coverage)	Can collapse	Good	Good	Good
Sampling speed	One pass, fast	One pass, fast	Many steps, slow	One step per token, slow
Likelihood available	No	A lower bound	A lower bound	Exact
Training	Unstable, needs care	Stable	Stable	Stable

Lesson: GANs compared with other generative models

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
292	Nim Game	Easy	You lose exactly when the number of stones is a multiple of 4	Whatever you take (1 to 3), your opponent can make the total of the round 4. So multiples of 4 are losing positions for the player to move. Two players who each play their best against the other is the same setting as the GAN game.
486	Predict the Winner	Medium	Minimax on intervals: best lead = max(take left minus their best lead, take right minus their best lead)	Let lead(i, j) be the most the player to move can get ahead by on nums[i..j]. Taking an end gives that number minus the opponent's best lead on what remains. This is a minimax game: each player assumes the other plays as well as possible, as the generator and the discriminator do.
877	Stone Game	Medium	Same interval minimax as Predict the Winner (and the first player always wins with an even count)	Compute the best lead of the player to move on piles[i..j] from shorter intervals. With an even number of piles and an odd total, the first player can always take all the even positions or all the odd ones, so the answer is always true, but the dynamic programming shows why.
375	Guess Number Higher or Lower II	Medium	cost(lo, hi) = min over guesses g of g + max(cost(lo, g minus 1), cost(g + 1, hi))	You pick the guess (min) and the number is chosen to hurt you most (max). Memoise the cost of every range. The min over your choices of the max over the opponent's choices is exactly the min over G of the max over D in the GAN value.
1406	Stone Game III	Hard	best[i] = max over taking 1, 2 or 3 stones of (stones taken minus best[next])	Work backwards from the end. The player to move takes 1 to 3 values and then the opponent faces the rest, so the lead is what you took minus the opponent's best lead. The sign of best[0] names the winner.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

In this tiny GAN the real numbers come from a bell curve with mean 4.00 and spread 0.50. With the original minimax loss the generator is still at mean 1.15 after 100 steps and its spread later collapses to 0.00. The non saturating loss reaches mean 3.83 by step 100 and settles at mean 4.03, but with a spread of only 0.17. When the discriminator can also see the spread of each batch, the generator ends at mean 4.00 and spread 0.50.

Lesson: The GAN game: a generator against a discriminator

IN THE EXAM

With the real data N(0, 1) and the generator N(m, 1), the best discriminator scores x = 0 as 0.500 when m = 0, 0.622 when m = 1 and 0.989 when m = 3. The GAN value V(D*, G) matches −ln 4 + 2 JS exactly: −1.3863 at m = 0, −1.1635 at m = 1, −0.3327 at m = 3 and 0.0000 at m = 10, where JS reaches its ceiling ln 2 = 0.6931.

Lesson: The best discriminator, the right loss, and mode collapse

IN THE EXAM

Shifting a uniform distribution on [0, 1] to the right, the JS divergence is 0.345 at a shift of 0.5 and then stuck at 0.693 for every shift of 1 or more, once the two stop overlapping. The Wasserstein distance keeps growing with the shift: 0.497, 0.994, 1.996, 4.997 and 9.996.

Lesson: Wasserstein GANs and other stable losses

IN THE EXAM

Using PCA features of the digits in place of Inception features, other real digits score 0.03 against the test digits. Adding noise with spread 0.2 gives 0.10 and spread 0.4 gives 0.33. A set of only real 1s scores 3.23, even though every image in it is a real digit: FID punishes mode collapse because the feature covariance is wrong. The average image repeated scores 4.04.

Lesson: DCGAN, conditional GANs, StyleGAN, and measuring quality